

CSC3043S - Assignment 1: Training a Transformer Language Model

Katlego Makhubedu - MKHKAT033

20 August 2026

AI Declaration

I used VS Code with Claude Code as my AI agent to write the code for this assignment. The repository's `AGENTS.md` file was kept in context for every session, so the agent worked to the conventions and constraints recorded there. All experiments were run by me on my own machine, and every number in this report comes from a log file committed to the repository. Claude Code also assisted in converting my report to latex, All conclusions, analysis and reasoning is my own work.

1 Implementation notes

Tokenizer. Byte-level BPE over the 256 byte values plus `<|endoftext|>`, using the GPT-2 pre-tokenizer regex, with ties broken by taking the lexicographically greatest pair. Merging is incremental: an index from each pair to the pre-tokens containing it means a merge only updates the counts it actually changes. Pre-tokenization runs over 8 processes on `<|endoftext|>`-aligned chunks. BPE is greedy, so the first k merges of a long run are exactly the merges a shorter run would learn - one 16,000-merge run therefore produces every tokenizer in this report by truncation.

Data. §2's test set is stored as its own file rather than sliced out at evaluation time (`valid_split.npy`: 25,630 documents, 5.24M tokens; `test_split.npy`: 2,000 documents, 409K tokens). Each split is encoded separately, because §6 needs the character count of exactly the text being scored and a token-index slice cannot give that. Every touch of the test set is recorded in `logs/test_set_ledger.json`, which holds one entry.

Model and training. Pre-norm blocks, RMSNorm, SwiGLU, RoPE, QK-norm, untied LM head, with `use_rmsnorm`, `use_rope` and `ffn_type` as CLI flags so each ablation is one flag away. AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$ and decoupled weight decay 0.1 applied only to parameters with $\text{ndim} \geq 2$; linear warmup into cosine decay to $0.1\eta_{\max}$, with the cosine period always set to the actual run length; gradient clipping at 1.0 with the pre-clip norm logged every step; bf16 autocast on GPU. Each arm of a comparison starts from a freshly initialised model, and step times are measured around the forward/backward/step only, with a CUDA sync on each side.

Evaluation. Perplexity is $\exp(L)$ over whole non-overlapping 256-token windows, and $\text{BPC} = L/(\ln 2 \times \text{chars-per-token})$ with C counted *excluding* the `<|endoftext|>` delimiters. The same procedure is used for every model compared.

Hardware. All GPU runs are on one NVIDIA GeForce RTX 4060 Laptop GPU (not an L4), PyTorch 2.13.0+cu130, Python 3.11.9. Tokenizer work ran on the same machine's 16-core Intel CPU. Every run is listed in Appendix A.

2 Task 1 - Tokenizer

Q1. *Wall-clock time to train and to encode, and the machine.*

Training BPE on the full 2.23 GB training file took **84.9 s** (65.5 s pre-tokenization over 8 processes, 19.4 s merging), and encoding that file took **531.1 s**, both on the 16-core Intel CPU. Both were timed at `vocab_size=16,000`, since every smaller tokenizer is a truncation of that merge list; encoding the training corpus with the chosen 4,000 tokenizer took 266.4 s.

Q2. *Tutorial vs. optimised BPE on a fixed 10 MB slice, vocab_size=1000.*

	Merge loop (s)	End-to-end (s)
Tutorial 1 (recount every merge)	9.39	10.45
Optimised (incremental counting)	0.26	1.31
Speedup	36.4×	8.0×

Table 1: Both implementations produce byte-identical merge lists. End-to-end includes the 1.05 s pre-tokenization pass that both share, which is why it is 8× rather than 36×: once merging is fast, pre-tokenization is the bottleneck.

Q3. *Compression ratio against vocabulary size, and the size chosen.*

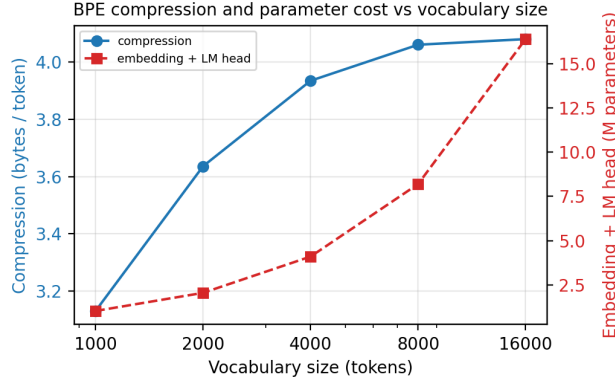


Figure 1: Bytes per token on the full training corpus (left axis), with the parameter cost of the embedding and LM head at $d_{\text{model}} = 512$ on the right. Source: `logs/vocab_study.csv`.

I chose **4,000**. Going from 1,000 to 4,000 improves compression from 3.13 to 3.94 bytes/token (+26%), but going on to 16,000 only adds another 3.7% while the embedding and LM head grow from 4.10M to 16.38M parameters. At 4,000 those two tables are 25% of the model; at 16,000 they would be 57%.

Q4. *Longest token, and the first and last five merges.*

Longest token in the 4,000 vocabulary: `b' uncomfortable'` (14 bytes). First five merges: `(' ', 't')`, `('h', 'e')`, `(' ', 'a')`, `(' ', 's')`, `(' ', 'w')`. Last five: `('u', 'r')`, `('in', 'ation')`, `('hang', 'ing')`, `('uc', 'et')`, `('ail', 'able')`. This makes sense: the early merges are the commonest short byte pairs in English, and by the end the common short pieces are already merged, so what is left to merge are whole word endings. The leading spaces show the GPT-2 pre-tokenizer attaching word boundaries to the token.

3 Task 2 - Model and inference

Q5. *Parameter count, and SwiGLU vs. parameter-matched ReLU.*

Component	SwiGLU ($d_{ff} = 1344$)	ReLU ($d_{ff} = 2048$)
Embedding (4000×512)	2,048,000	2,048,000
LM head (untied)	2,048,000	2,048,000
Non-embedding	12,456,960	12,588,032
Total	16,552,960	16,684,032

Table 2: The two FFN variants differ by 131,072 parameters, +0.79%, inside §7.2’s 2% tolerance. §4.1’s “roughly 17M excluding the embedding and LM head” does not match this model: the non-embedding count is 12.46M, and 16.55M is the total including both tables.

Q6. *Evidence the KV cache is correct.*

Over a 100-token greedy continuation the cached and uncached paths produced identical token sequences, and the maximum absolute logit difference was 1.49×10^{-6} (mean 1.08×10^{-6}), well inside the 10^{-4} tolerance.

The two paths also chose the same token at every step, not just the same final sequence, which is what catches a RoPE-applied-at-the-wrong-position bug.

Q7. *Generation throughput with and without the KV cache.*

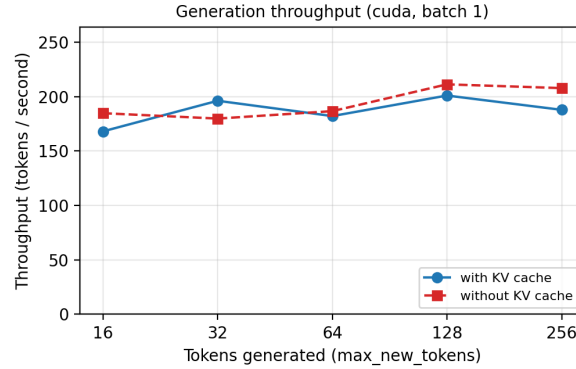


Figure 2: Tokens/second against tokens generated, batch 1, mean of 3 trials. Throughput uses the tokens actually produced, not `max_new_tokens`: the cached path stops when it fills its `context_length` buffer, so at 256 it produced 246 tokens against the uncached path’s 256.

At 256 tokens the speedup is $0.90\times$ — the cache was about 10% *slower*, and both curves are flat instead of separating. The model is small and the context is short, so each decode step spends most of its time on the fixed cost of launching GPU kernels rather than on attention. Recomputing the prefix is therefore almost free at these lengths, while the cache adds indexing and buffer writes on every step. The cache should win once the context is long enough for attention to dominate the step.

4 Task 3 - Training

Q8. *Mean step time in fp32 and bf16, and what stays in fp32.*

Over 200 steps per arm at batch = 32×256 , discarding the first 20: **fp32 240.17 ms** and **bf16 autocast 112.86 ms**, a $2.13\times$ speedup. Autocast runs the matrix multiplications in bf16 — every `nn.Linear`, the attention scores, the FFN, the LM head — and keeps softmax, cross-entropy, `rsqrt` and other reductions in fp32. This matters because bf16 only carries about 8 bits of precision. All three of these operations sum many values together: the softmax normaliser adds thousands of exponentials, RMSNorm averages 512 squared activations before taking a reciprocal square root, and cross-entropy takes the log of a 4,000-way normaliser. Rounding error accumulates across such a sum, and it would feed straight into the loss and the gradients, so those parts stay in fp32.

Q9. *Best learning rate with and without warmup.*

Effect of warmup at $\text{lr}_{\text{max}} = 3\text{e-}3$ (2,000 steps)

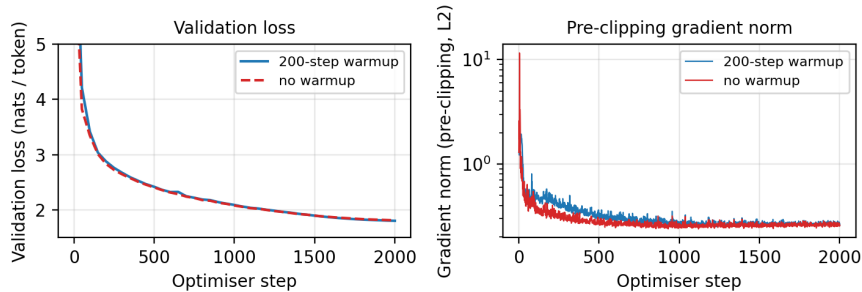


Figure 3: 2,000 steps at $\eta_{\text{max}} = 3 \times 10^{-3}$, everything else fixed, both arms from identical initial weights. The right panel is per-step, since an every-50-steps log samples straight past the effect.

Run	grad norm @1	@50	@2000	max over run	final val loss
200-step warmup	1.2942	0.4930	0.2658	1.9320	1.8057
No warmup	1.2942	0.4366	0.2586	11.5899	1.8141

Table 3: Pre-clipping gradient norms. Step 1 is identical because both arms start from the same weights and the same batch.

The two validation curves are almost identical and warmup wins by only 0.0084 nats. The difference shows up in the gradient norm instead: without warmup it spikes to **11.59** in the first few steps, more than $11\times$ the clipping threshold, so those early updates were set by the clip rather than by the optimiser. With warmup the maximum over the whole run is 1.93. At this learning rate clipping was enough on its own, so warmup made little difference here, but it is cheap protection against a spike large enough that clipping alone would not recover.

5 Task 4 - Experiments

All runs use the §7 standard configuration — 4 layers, $d_{\text{model}} = 512$, 8 heads, $d_{ff} = 1344$, context 256, batch 32, 5,000 steps, 200 warmup, cosine to $0.1\eta_{\text{max}}$ with the period matched to the run, clip 1.0, bf16, seed 42 — and differ only in the one thing named.

Q10. Learning-rate sweep.

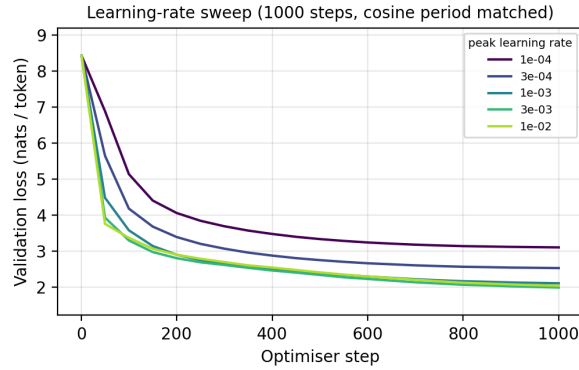


Figure 4: Five peak learning rates over two orders of magnitude, run at 1,000 steps with the cosine period shortened to match (§7.1). Final validation loss: 10^{-4} : 3.100; 3×10^{-4} : 2.526; 10^{-3} : 2.104; 3×10^{-3} : **1.987**; 10^{-2} : 2.034.

My best learning rate is 3×10^{-3} , and every run after this uses it. **No run in this grid diverged**, which is a gap against §7.1 — the largest rate I tested, 10^{-2} , trained fine and came second. So all I can say is that divergence starts somewhere above 10^{-2} . I should have added a run at 3×10^{-2} , which would have cost about two minutes. The sweep does show that the two directions are not symmetric: being $10\times$ too low costs +0.54 nats, while being $3\times$ too high costs +0.047. On a new model it is therefore better to start from a working rate and raise it until training breaks, rather than starting low and creeping up. I do have a divergent run, but from the ablations: the no-RMSNorm model diverged at this same 3×10^{-3} (Q11) and was killed automatically at step 150.

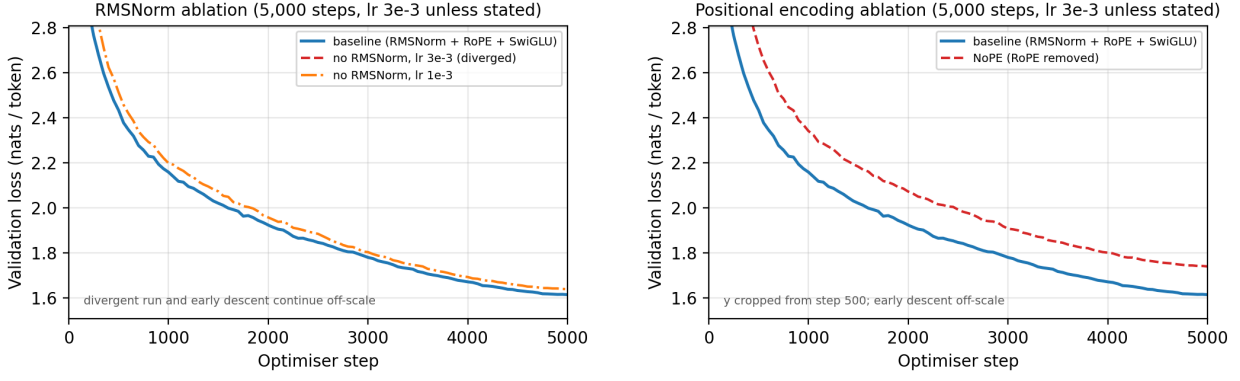


Figure 5: Left (Q11): RMSNorm ablation. **Right (Q12):** RoPE removed. Both share the y-range with Figure 6 so the ablations can be compared directly. The range starts from step 500, because every run begins at $\ln 4000 = 8.29$ and including that flattens all the curves into one line.

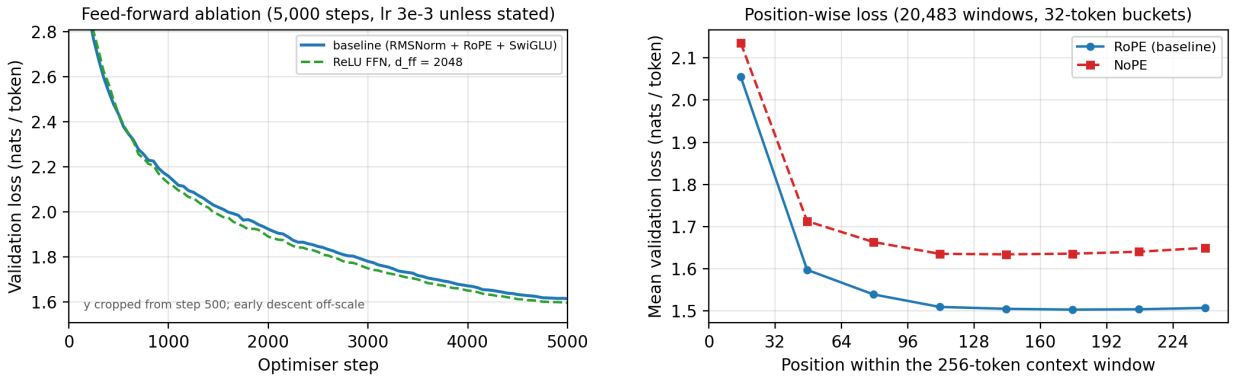


Figure 6: Left (Q13): SwiGLU against the parameter-matched ReLU FFN. **Right (Q17):** mean validation loss by position in the context window, 32-token buckets, 20,483 windows.

Q11. *No RMSNorm, at the best and at a reduced learning rate.*

At 3×10^{-3} the no-RMSNorm model **diverged at step 150** (validation loss 131.45, against 9.84 at step 1). At 10^{-3} it trained normally and reached 1.6386 against the baseline’s 1.6140, a gap of only **+0.0246 nats**. So RMSNorm is not adding much to what the model can represent — it is keeping training stable. What it buys is the ability to use a $3\times$ larger learning rate without activations growing out of control, and that larger rate is worth 0.117 nats in the sweep, far more than the 0.025 the normalisation itself costs.

Q12. *NoPE against the RoPE baseline.*

Removing RoPE costs **+0.1253 nats** (1.7393 against 1.6140), the largest gap of any ablation that trained. That the model still works is not surprising, because the causal mask already carries some positional information: token i can attend to more previous tokens than token $i - 1$, so the model can partly work out where it is from the size of its own attention window. The size of the gap is the interesting part — it is $7\times$ the SwiGLU-to-ReLU gap and $2.7\times$ the gap between my best and second-best learning rate, so reconstructing position from the mask alone is expensive.

Q13. *SwiGLU against the parameter-matched ReLU FFN.*

The ReLU FFN at $d_{ff} = 2048$ reached **1.5961** against SwiGLU’s 1.6140, so SwiGLU **lost by 0.0179 nats**. The two are matched on parameter count to within 0.79%, but not on anything else: ReLU uses two larger matrices where SwiGLU uses three smaller ones. The gap is also small — about 14% of the NoPE gap, and from a single seed — so the fair conclusion is that at this scale SwiGLU gives no measurable benefit over a parameter-matched ReLU, not that ReLU is better.

Q14. *The design choices side by side.*

Design choice	Comparison	Δ val loss	Note
RMSNORM	baseline vs. removed, at 3×10^{-3}	diverged	killed at step 150
RoPE	baseline vs. NoPE	+0.1253	largest gap among runs that trained
Learning rate	3×10^{-3} vs. 10^{-2}	+0.0470	best vs. second-best in the sweep
SwiGLU	baseline vs. parameter-matched ReLU	-0.0179	ReLU was better

Table 4: Positive Δ means the baseline’s choice was better. Baseline final validation loss 1.6140.

Normalisation mattered most, since without it the best configuration does not train at all, and positional encoding was the only architecture choice worth a measurable amount of loss. Getting the learning rate right to within a factor of three was worth $2.6\times$ as much as the whole SwiGLU-versus-ReLU decision. At this scale the effort is better spent on tuning and stability than on the feed-forward block.

Q15. *Vocabulary size: 4,000 against 1,000.*

Run	Vocab	Parameters	Val loss	Perplexity	BPC
baseline	4,000	16,552,960	1.5896	4.902	0.5857
vocab_1000	1,000	13,480,960	1.3461	3.842	0.6226

Table 5: Both scored on `valid_split.npy` over whole non-overlapping 256-token windows, C excluding `<|endoftext|>`. Chars per token: 3.9156 and 3.1191; 5.24M and 6.58M tokens scored over the same text.

The 4,000-vocabulary model is better. Perplexity says the opposite, but the two models are not predicting the same things: the 1,000 model makes 6.58M easier predictions where the other makes 5.24M harder ones over identical text, so their perplexities are not comparable. BPC normalises by the text instead of by tokens, and on BPC the 4,000 model wins, 0.586 against 0.623.

The parameter count makes this less clean. The 4,000 model has 3.07M more parameters (+23%), all of them in the embedding and LM head, so it is not purely a test of the tokenizer. But those extra parameters only make each prediction harder, not easier, and the model still wins on BPC. It also gets 26% more text through each step at the same batch size, which is the main advantage at a fixed step count.

Q16. *Compute budget.*

Phase	Runs	Steps	Wall-clock (s)	GPU-hours
Development / debugging (CPU + smoke runs)	—	—	not instrumented	—
Precision timing, Q8	2	400	71	0.02
Warmup study, Q9	2	4,000	452	0.13
Learning-rate sweep, Q10	5	5,000	614	0.17
Baseline	1	5,000	630	0.18
Ablations, Q11–Q13	4	15,150	1,665	0.46
Vocabulary study, Q15	1	5,000	590	0.16
Final model, §7.4	1	20,000	2,504	0.70
Total	16	54,550	6,526	1.81

Table 6: From the `wall_seconds` field of each `logs/*_run.json`; Q8 and Q9 have no run record and are computed from their logged step times. These are RTX 4060 Laptop hours, not L4 hours. Development time was not instrumented — debugging ran on CPU and on the small configurations in `logs/smoke/`. The GPU-hours column is rounded per row, so it sums to 1.82 against a total of 1.81 computed before rounding.

I would spend a doubled budget on **more steps** for the final model, not on more configurations. At 20,000 steps the model has seen 163.8M tokens, which is only **0.29 of one epoch** of the 559.8M-token training set, and its training loss (1.4204) is still slightly above its validation loss (1.4157), so it is not overfitting. The $4\times$ step increase from baseline to final model already bought 0.198 nats, more than any architecture choice in Q14.

Q17. *Loss by position within the context window.*

(Figure 6, right.)

Positions	0–31	32–63	64–95	96–127	128–159	160–191	192–223	224–255
RoPE (baseline)	2.0549	1.5971	1.5388	1.5090	1.5045	1.5027	1.5035	1.5067
NoPE	2.1355	1.7127	1.6631	1.6350	1.6340	1.6354	1.6400	1.6492
Gap	0.0805	0.1156	0.1244	0.1260	0.1294	0.1327	0.1366	0.1425

Table 7: Mean validation loss (nats/token) over 20,483 non-overlapping windows.

The single number in Q12 hides the shape of the problem. The gap is not constant: it grows steadily with position, from 0.081 in the first bucket to 0.143 in the last. The RoPE model flattens out after about position 96 and stays flat, which is what you want — more context neither helps nor hurts once the model has enough. The NoPE model instead reaches its lowest loss around position 128 and then gets *worse* for the rest of the window. Its only clue about position is how many tokens it can attend to, and that clue weakens as the window fills: the difference between 200 and 201 previous tokens says far less than the difference between 10 and 11. So NoPE degrades exactly where the model should be improving, and the gap would likely be larger at longer context lengths.

6 Task 5 - Final model and generation

The final model is the baseline configuration at $\eta_{\max} = 3 \times 10^{-3}$ trained for 20,000 steps ($4\times$ the standard run), with the cosine period set to the full length: 16,552,960 parameters, 163.8M tokens processed, 2,504 s.

Q18. *Final model on validation and on test.*

Split	Tokens	Windows	Loss (nats/token)	Perplexity	BPC
Validation (<code>valid_split.npy</code>)	5,243,648	20,483	1.3978	4.046	0.5150
Test (<code>test_split.npy</code>)	408,832	1,597	1.4166	4.123	0.5218

Table 8: Whole non-overlapping 256-token windows, C excluding `<|endoftext|>`. The test set was evaluated once, recorded in `logs/test_set_ledger.json`.

Test loss is 0.0188 nats above validation (+1.3%), which is expected given the test split is $13\times$ smaller; nothing was selected on it. The $4\times$ step increase moved validation BPC from 0.5857 (baseline, Q15) to 0.5150.

Q19. *256 tokens from “Once upon a time,” under three decoding settings.*

Full texts are in `samples.txt`. Every sample stopped on `<|endoftext|>` before reaching 256 tokens.

(a) **Greedy**, $T = 0$ — 182 tokens, distinct-token ratio 0.44:

Once upon a time, there was a little girl named Lily. [...] Lily ran to her mom and said, “Mom, can I play with the big, red ball?” Her mom said, “Yes, but be careful. The ball might fly away.” [...] They got a long stick and got the ball down. Lily was happy again.

(b) $T = 0.8$, **top-k** = 40 — 115 tokens, distinct-token ratio 0.57:

Once upon a time, there was a little boy named Tom. Tom loved to climb. [...] One day, Tom saw something weird outside the tree. It was a big, shiny rock. Tom did not know what it was. He was very curious. Tom climbed back down and found a small, shiny rock. He was so happy!

(c) $T = 0.9$, **top-p** = 0.95 — 193 tokens, distinct-token ratio 0.46:

[...] As he climbed, he saw a little bird outside. The bird looked sad. Tom wanted to help the bird, so he went inside. Tom’s mom saw the bird too. She was very worried. Tom climbed down the tree and found out he could help the bird.

I would ship (b), $T = 0.8$ **with top-k** = 40: it has the most varied wording of the three (distinct-token ratio 0.57 against 0.44 and 0.46) and is the only one that stays consistent from start to finish — (a) is fluent but formulaic, and in (c) the model goes “inside” to help a bird that is outside.

Q20. *A degenerate decoding setting, and a failure mode that is not a decoding artefact.*

Decoding — a negative result. My candidate setting was $T = 0.15$ with no truncation, and it did *not* degenerate. Its distinct-token ratio is 0.49, slightly *higher* than the recommended sampler’s 0.46, and it ended cleanly on `<|endoftext|>` after 123 tokens.

Lily kicked the ball very hard. The ball went up, up, up into the sky. Lily was sad. She wanted her ball back. She thought of a plan. / Lily ran inside her house. She found a big box.

Low temperature does not produce a loop here because TinyStories documents are short and end in a strongly-predicted `<|endoftext|>`, so near-greedy decoding follows the single most likely story and finishes before a loop can start. I have no logged sample that is degenerate from decoding alone. I would expect one at the opposite extreme, $T \geq 1.5$ with no top- k or top- p , where the whole 4,000-token tail is sampled often enough to break the grammar, but I did not run it.

Model — repetition and no memory of what it has said. This one is not caused by the decoding settings: it appears at the recommended $T = 0.9$, top- $p = 0.95$ across different prompts (mean distinct-token ratio 0.449 over three), and truncation sampling does not remove it. From “Tom and Sara wanted to build”:

They found some tape, some tape, and some soap. [...] They only wanted to use the tape. They did not listen to Mom. They kept knocking and tugging. They did not listen to Mom.

The model repeats a whole sentence four sentences later and lists “some tape” twice in one sentence, because nothing in it keeps track of what it has already written — it only predicts a plausible next token from the last 256. The same weakness shows up as contradiction rather than repetition elsewhere: the children are told to clean up and then immediately put on their pyjamas, and in Q19(c) Tom goes inside to help a bird that is outside.

A Experiment log

Every run launched, including the ones that failed. Wall-clock and losses come from `logs/<name>_run.json`. Validation loss is the value at the run’s final step, which differs slightly from the whole-corpus re-evaluation quoted in Q15.

Run	Configuration	Steps	Final val	Wall (s)	GPU-h	What I learned
lr_sweep_1e-04	$\eta = 10^{-4}$, 1k steps	1,000	3.1000	124	0.03	far too low
lr_sweep_3e-04	$\eta = 3 \times 10^{-4}$	1,000	2.5255	123	0.03	still too low
lr_sweep_1e-03	$\eta = 10^{-3}$	1,000	2.1039	123	0.03	workable, not best
lr_sweep_3e-03	$\eta = 3 \times 10^{-3}$	1,000	1.9868	123	0.03	best; used everywhere after
lr_sweep_1e-02	$\eta = 10^{-2}$	1,000	2.0339	123	0.03	second best; <i>did not diverge</i>
q8_fp32	fp32, timing only	200	—	48	0.01	240.17 ms/step
q8_bf16	bf16 autocast, timing only	200	—	23	0.01	112.86 ms/step; $2.13\times$
q9_with_warmup	200 warmup steps	2,000	1.8057	226	0.06	max grad norm 1.93
q9_no_warmup	no warmup	2,000	1.8141	226	0.06	grad norm spikes to 11.59
baseline	§7 standard, $\eta = 3 \times 10^{-3}$	5,000	1.6140	630	0.18	comparison point for the ablations
ablation_no_rmsnorm	-no_rmsnorm, $\eta = 3 \times 10^{-3}$	150	131.45	16	0.00	diverged ; killed at step 150
ablation_no_rmsnorm_lowlr	-no_rmsnorm, $\eta = 10^{-3}$	5,000	1.6386	511	0.14	trains fine; RMSNorm buys stabili
ablation_nope	-no_rope	5,000	1.7393	551	0.15	largest ablation gap, +0.125
ablation_relu	-ffn_type relu -d_ff 2048	5,000	1.5961	587	0.16	beat SwiGLU by 0.018
vocab_1000	second tokenizer, vocab 1,000	5,000	1.3567	590	0.16	lower loss, worse BPC
final_model	baseline, $4\times$ steps	20,000	1.4157	2,504	0.70	still improving at the end

Table 9: 16 runs, 1.81 GPU-hours on an RTX 4060 Laptop GPU. Rows are rounded to the nearest second, so they can differ slightly from the phase totals in Q16, which are summed before rounding. The divergent run cost 16 seconds because it was killed automatically; left to finish it would have cost about 520.

Runs I did not launch, and should have. A sweep point at 3×10^{-2} to find where divergence starts (Q10, about 2 minutes of GPU time), and a high-temperature untruncated generation for Q20 (about 1 second).

B Reproducing the figures

Every figure is regenerated from `logs/` alone by `python scripts/make_plots.py`.

Figure 1 $\leftarrow$ `logs/vocab_study.csv`. Figure 2 $\leftarrow$ `logs/task2_results.json`. Figure 3 $\leftarrow$ `logs/q9_{with,no}_warmup{,_s`
Figure 4 $\leftarrow$ `logs/lr_sweep*.csv`, `logs/task4_results.json`. Figures 5 and 6 $\leftarrow$ `logs/baseline.csv`,
`logs/ablation*.csv`, `logs/task4_results.json`. Tables come from `logs/task{1,2,3,4,5}_results.json`
and `logs/*_run.json`.